

A Knowledge-Based Agent for ARC-AGI

Asif Rasool
asif.rasool@gatech.edu

1 AGENT DESCRIPTION

My final ARC-AGI agent is a symbolic solver that treats each problem as a search over explicit, interpretable explanations. The agent analyzes the training pairs, retrieves relevant symbolic transformation families, fits and tests candidate hypotheses, ranks the surviving explanations, and then applies the strongest ones to the test input. Appendix A includes a flowchart that summarizes this end-to-end process.

1.1 Step 1: Extract structural features from the training pairs

For each training example, the agent records input and output shapes, color sets, background colors, component counts, density, separator axes, and whether the output already matches a non-background bounding box from the input. The agent uses those measurements to build a coarse problem frame, such as shape-preserving transformation, extraction, composition, or object-relational reasoning, which narrows the search space early.

1.2 Step 2: Retrieve the most plausible concept families

After feature extraction, the agent queries a registry of symbolic concepts. A concept is a hand-designed solver for a recognizable transformation pattern. Some concepts are direct solvers for relatively specific operations such as rigid transforms, cropping, recoloring, panel combination, or object-level manipulation. Other concepts are broader families that generate multiple related symbolic candidates for harder tasks such as marker paths, center connection, motif completion, legend-based extraction, frame symmetry, and enclosure repair. Each concept assigns its own hand-written applicability score from the extracted problem features, and the registry ranks concepts by those scores before passing only the positive-scoring ones into the next stage.

1.3 Step 3: Fit concrete hypotheses within those concept families

Once the candidate concepts are chosen, the agent fits concrete hypotheses to the training pairs. Each hypothesis is a fully specified symbolic explanation with parameters and a deterministic `predict()` routine. Earlier versions of the agent

often tried to produce one fitted explanation per concept. The final version is more flexible: many concepts can generate several fitted variants with different parameter settings instead of forcing one immediate interpretation. ARC tasks often admit several locally plausible interpretations, so keeping multiple symbolic parameterizations alive makes the solver less brittle and gives it a better chance of preserving the right abstraction when one narrow interpretation breaks.

1.4 Step 4: Validate hypotheses on the visible training examples

After fitting, the agent checks each hypothesis against the training input-output pairs. Exact matches are kept as primary candidates. Near-miss hypotheses are not discarded immediately; instead, the system records an explicit training-error count for them, since a near-miss can still contain the right idea but the wrong parameter choice. The agent also runs a bounded pipeline search over short symbolic scripts built from primitive operations such as cropping, color swapping, color mapping, and rigid transforms, which is useful for tasks whose best explanation is a short composition rather than one isolated concept.

1.5 Step 5: Rank the surviving explanations

Each fitted candidate receives an explanation signature describing its shape behavior, frame, and structural assumptions. A ranker combines frame match, case-based analogy, residual training error, confidence, and complexity cost into a diagnostic score. Case memory is used only within a single problem: once the agent finds a strong explanation, it can slightly favor structurally similar explanations for that same task. [Appendix E](#) gives a more explicit description of how that ranking procedure works.

1.6 Step 6: Generate final predictions and targeted repairs

The last step is to turn the ranked hypotheses into up to three final predictions. The agent first tries to diversify its outputs across concept families so the three predictions represent meaningfully different explanations.

The agent also generates targeted repair candidates for concept families that often produce plausible but fragile solutions. A repair candidate is a structured modification of an already promising hypothesis, usually created by changing one small symbolic choice such as the selected center, frame, path-routing rule, symmetry mode, cone pattern, motif completion rule, or legend-object interpretation. Repair generation is especially important for families such as center connection,

frame symmetry, motif completion, legend-object extraction, marker paths, enclosure repair, spirals, and slide-to-border behavior. Appendix F describes that repair process in more detail.

If exact candidates do not fill all three output slots, the solver next considers repaired hypotheses and then near-miss candidates that still encode a reasonable symbolic explanation. Only if none of those sources produce a usable prediction does it fall back to a default answer.

2 AGENT PERFORMANCE

The final agent performed strongly across the full ARC-AGI benchmark used in the course project. According to the final Gradescope results in Appendix B, it solved 48 out of 48 visible training problems and 45 out of 48 hidden test problems, for a total of 93 out of 96 correct. The result corresponds to 100% accuracy on the visible training set, 93.75% accuracy on the hidden test set, and 96.88% overall accuracy across all 96 evaluated problems. The full Gradescope run completed in 19.634008 seconds.

2.1 Overall Results

Table 1 summarizes the overall results at the level of visible training problems, hidden test problems, and total solved problems.

Table 1—Overall final-project performance summary.

Metric	Result
Visible training problems passed	48 / 48
Hidden test problems passed	45 / 48
Total problems passed	93 / 96
Overall running time	19.634008 seconds

2.2 Set-by-Set Breakdown

Gradescope also reports results by milestone set. Since each milestone contains 16 visible training problems and 16 hidden test problems, the final report can separate performance cleanly across Sets B, C, and D. Table 2 shows that the final agent solved every visible problem in all three sets. On the hidden side, it missed one Test B problem, no Test C problems, and two Test D problems.

Table 2—Set-by-set performance across the full project benchmark.

Set	Training	Hidden Test	Total
B	16 / 16 (100%)	15 / 16 (93.75%)	31 / 32 (96.88%)
C	16 / 16 (100%)	16 / 16 (100%)	32 / 32 (100%)
D	16 / 16 (100%)	14 / 16 (87.50%)	30 / 32 (93.75%)

Set C was the strongest part of the benchmark, while Sets B and D exposed the remaining hidden-test failures. Even so, the architecture performed at a high level across all three sets and remained perfect on the visible training tasks.

3 SUCCESSFUL PROBLEMS

3.1 Problem 0520fde7: separator-based Boolean composition

Problem 0520fde7 is one of the easiest examples to explain because the main visual cue is very clear. Each input has a vertical separator that splits the grid into a left panel and a right panel. The output is much smaller than the input, so the task is “compare the two sides and produce a compact summary.” The full family appears in Appendix C as Figure 2.

My agent begins by recording simple structural facts: the separator is uniform, the input contains two aligned regions, and the output shrinks dramatically. The features push the problem toward the composition family rather than toward rigid transforms, cropping alone, or object movement. Retrieval then brings forward concepts designed for combining two subgrids. For this problem, the top exact explanation is `SplitBoolean`.

`SplitBoolean` treats the left and right panels as aligned masks and checks a Boolean relation cell by cell. The agent asks: when the two sides are compared position by position, which cells should survive in the output? Training validation succeeds because the same comparison rule works on every visible example even though the final small pattern changes from case to case.

Success here comes from a close match between the problem and the agent’s design. Clean separators and compressed outputs are strong signals for panel composition in my system. A human would probably also start by saying, “the divider matters, so the rule must compare the left side to the right side.” My agent reaches the same conclusion in a narrower way: it selects from a small set of

Boolean panel-combination rules and ranks the one that exactly fits the training pairs.

3.2 Problem 62c24649: generative symmetry from a small seed

Problem 62c24649 shows a very different kind of success. Each input is a small seed pattern, while each output is a larger, highly regular design. Nothing is being cropped out of the input, and the result is not a simple recoloring. The important fact is that the output grows while preserving a strong symmetric structure. Appendix C shows the full examples in Figure 3.

My agent reads that growth pattern early. Feature extraction marks the task as generative because the output is larger than the input, and it also records the regular repeated structure in the outputs. Retrieval then favors concepts that can build a larger design from a compact starting pattern. For this family, the best exact explanation is `QuadrantMirror`.

Under `QuadrantMirror`, the input is treated as a seed that should be reflected across horizontal and vertical axes. Once the correct seed is chosen, the rest of the output follows from a fixed procedure: mirror rows, mirror columns, and place the reflected copies into a larger arrangement. Training validation is easy for the agent because one simple symmetric rule explains all visible examples.

Clarity of the underlying invariant is the reason this problem is solved reliably. A human might summarize the rule as “take the small pattern and mirror it into a bigger balanced shape.” My agent solves the task in almost the same conceptual way, but it stores the idea as a deterministic symbolic operation rather than as an intuitive visual pattern. Problem 62c24649 therefore illustrates that the final system can do more than compare panels or extract objects; it can also generate a larger structured output from a small input when the symmetry cue is strong enough.

3.3 Problem e9b4f6fc: legend-guided extraction and recoloring

Problem e9b4f6fc is the best example of the more relational reasoning added late in the project. Each input contains a large multicolor object plus small detached color pairs elsewhere in the grid. The output is not the whole scene. The output is a cropped version of the main object after its colors have been changed. Visual inspection suggests that the small detached pairs are not noise; they act like a

legend that tells the solver how to recolor the object. Figure 4 in Appendix C shows the full family.

My agent handles the problem in several steps. Feature extraction first notices that the output is smaller than the input, so extraction is likely involved. Retrieval then favors legend-based object concepts instead of plain cropping or plain recoloring rules. The top exact explanation is `LegendPairRecolorExtract`, and several `LegendObjectLibrary` variants also fit the visible examples exactly.

The winning explanation does three jobs in sequence. First, it identifies the main non-background region that should be preserved. Second, it reads the small detached color pairs as a mapping from source colors to target colors. Third, it crops the relevant object and applies the inferred color mapping. Training examples support that explanation because the detached marker pairs change from case to case, yet the same “read the legend, then recolor the main object” procedure still works.

Problem `e9b4f6fc` shows why the final agent needed concept families and ranking. More than one legend interpretation can be made to fit the visible training pairs. The final system succeeds because the ranker prefers the explanation whose object choice, extraction behavior, and recoloring logic all line up at once. A human would probably describe the task as “use the small color key to repaint the big object.” My agent reaches the same answer, but through explicit symbolic steps.

4 FAILURE ANALYSIS

4.1 Problem `28e73c20`: brittle spiral generation

Problem `28e73c20` shows a failure from Set B. Every visible training input is nearly empty, and every visible output is a spiral-like enclosure drawn into that empty space. Figure 5 in Appendix D makes the problem clear: the agent does not need to find existing objects in the input, because the input contains almost no information except the grid size. The real challenge is to infer exactly which spiral should be generated.

My agent does reach the correct general family. Feature extraction classifies the task as largely generative, and the top visible explanation is `SpiralGenerator`. The problem is not concept retrieval. The problem is that several different spiral variants can fit the visible examples when the inputs are so constrained. A

solver can vary the turn direction, the starting location, the corridor width, or the wall/background interpretation and still match the training cases.

The hidden miss therefore comes from ambiguity inside the spiral family. Step 3 of the pipeline generates a concrete spiral hypothesis, and Step 5 ranks that hypothesis against nearby alternatives. On the visible examples, the selected variant looks correct. On the hidden case, a different spiral parameterization was needed. Put differently, the agent learns “draw a spiral,” but it does not learn the deeper geometric intention strongly enough to choose the one spiral that generalizes.

A human solver would probably describe the task at a more higher level, such as “keep one continuous path and curl it inward in the same way every time.” My agent does not store that kind of geometric description directly. It chooses among a small set of symbolic spiral variants. Problem 28e73c20 therefore reveals a weakness in hypothesis fitting and ranking when the visible examples do not constrain one unique generative rule.

4.2 Problem 60a26a3e: center selection under distractors

Problem 60a26a3e shows a different failure from Set D. The visible examples contain several red diamond-like groups. The correct output adds blue lines that connect selected centers by rows and columns. Figure 6 in Appendix D shows the family. At a high level, the task is not hard to name: find the right centers, then connect them. The difficult part is deciding which apparent centers are real and which ones are distractors.

My agent again reaches the correct concept family. The top visible hypotheses are `ConnectDiamondCenters` and `CenterConnectionLibrary`. The solver therefore understands that center-connection reasoning is needed. Hidden failure appears later in the pipeline. Before any blue path can be drawn, the agent must choose the anchor points. A wrong anchor choice immediately ruins the whole output, because the connection rule itself is deterministic once the anchors are fixed.

Local testing in my repository suggests that the final version solved many variants with extra false centers, rings, noise, and other distractors. Remaining failure on the real hidden case means one distractor pattern still looked plausible enough to survive the agent’s center-selection rules. Repair generation in Step 6 tries several alternatives, such as row-only, column-only, and different gap settings, but all of those repairs inherit the same original center-identification error. As a result,

the agent searches around the wrong starting point instead of correcting the real source of the mistake.

Problem 60a26a3e therefore exposes a weakness in perceptual selection. A human would probably reject false centers by using a richer sense of symmetry, grouping, and visual prominence across the whole scene. My agent uses a much narrower symbolic test for what counts as a valid center. When variants closely mimic the target pattern, the right concept can still be chosen and the final answer can still be wrong.

5 AGENT DEVELOPMENT

My overall design approach was iterative. I kept the same central idea for the whole project: ARC problems should be solved by searching over explicit symbolic explanations. The main design question was how a symbolic agent could search more intelligently among plausible explanations.

At a high level, my agent is an ever-expanding heuristic system, but calling it only a list of heuristics would miss an important part. A flat collection of rules is too brittle for ARC because many tasks can look compatible with several local patterns at once. For that reason, I gradually reorganized the agent so it could do more than match a problem to one stored trick. The final design extracts structural features, uses those features to narrow the search space, generates several symbolic hypotheses, validates them on the training pairs, and ranks them according to how well they fit the overall structure of the task. In other words, the heuristics became inputs to a search procedure instead of isolated answers.

The biggest change in my thinking was how to handle ambiguity. Around the early milestones, most of the agent was a direct inventory of rules: rigid transforms, cropping, recoloring, and a few panel operations. Milestone C pushed me toward more compositional reasoning because many tasks needed extraction plus recoloring or multiple panels combined into one output. Milestone D pushed the agent further toward object selection and repair because hidden-style variants often used the right broad pattern but changed one local detail. Early versions implicitly assumed that one clean concept should explain the whole task and that anything short of an exact fit probably was not useful, but that assumption turned out to be too strong. The final agent therefore keeps multiple hypotheses alive, allows short symbolic pipelines when one rule is not enough, and uses concept families for patterns that are too variable to be captured by one narrow solver.

Another important part of the development process was changing how I treated near misses. I noticed that exact-match filtering by itself was too brittle. A near miss can still encode the right abstraction with one symbolic choice set incorrectly. The realization led to a ranker that considers structural fit, analogy, confidence, residual training error, and complexity together instead of asking only whether a candidate is exact and simple; Appendix E describes that ranking procedure in more detail. It also led to repair generation, where the agent modifies a promising explanation by changing one decision at a time, such as a center choice, a path rule, a symmetry mode, or a legend mapping; Appendix F explains that process.

The agent did expand over time from a small direct concept inventory to a larger final system with concept families, pipeline search, ranking, and repair. The high-level story is that I started with symbolic rule matching and kept pushing it toward symbolic hypothesis search.

6 AGENT-HUMAN COMPARISON

My final agent is similar to how I think I would approach ARC-AGI as a human at a high level, but not at the level of flexibility. In both cases, the first step is to look for the important structure in the examples instead of thinking cell by cell. If I were solving an ARC task myself, I would immediately ask what the relevant objects are, whether the output preserves shape or size, whether some region is acting like a separator, and whether one small pattern is serving as a cue for what happens elsewhere. My agent does something very similar. It measures structural features of the training pairs first and uses those features to decide which symbolic explanations are worth considering. In that sense, both the human and the agent begin by trying to find the right way to look at the problem before committing to an answer.

Another major similarity is that both the human and the agent work by forming and testing possible explanations. When I solve these problems by hand, I do not usually jump straight to one answer with complete certainty. I generate a few plausible interpretations, check them against the visible examples, throw out the ones that fail, and keep refining the ones that survive. My final agent follows almost the same outer pattern. It retrieves plausible concept families, fits several hypotheses, validates them on the training pairs, ranks the survivors, and then returns up to three predictions. A human might think, “this looks like a symmetry problem” or “the small shape is probably a key.” My agent reaches the same kind

of intermediate conclusion in a more rigid way by matching the problem against hand-built symbolic families.

The biggest difference is that a human can change representations much more freely. If my first interpretation of a task feels too narrow, I can quickly reframe it. I might first see a problem as object manipulation, then realize it is really a color mapping problem, then realize it is actually driven by a more global pattern. My agent is not nearly that flexible. It can only search among the symbolic concepts, short pipelines, and repair rules that I built into it. If the right answer depends on a representation outside that library, the agent cannot invent a fundamentally new concept on the spot. It can only search nearby variants of what it already knows. For that reason, the remaining failures tend to be problems where the visible examples leave several plausible interpretations open or where the important cue is easy to confuse with a distractor.

Another difference is that a human uses broader judgment when deciding which explanation is most likely to generalize. Even if two rules fit the visible examples, I can often reject one because it feels too specific or too dependent on an accidental detail of the training cases. My agent has only a limited version of that ability. It does rank explanations, preserve multiple hypotheses, and generate repair candidates when a top answer looks fragile, but that is still much narrower than human judgment. I can step back and ask whether I am using the wrong overall strategy. The agent cannot truly reflect on its own knowledge base and decide that it needs a new kind of concept during the same problem.

Overall, I would say the final agent approaches ARC in a way that is genuinely similar to my own process in outline but much narrower in execution. The two are similar because both try to identify structure, form candidate explanations, test those explanations against the examples, and keep ambiguity alive when the problem is not fully determined. They differ because a human can shift abstraction levels more fluidly, bring in broader visual intuition, and invent new ways of interpreting a task when the obvious explanation fails. My final agent is therefore similar to a human in strategy, but not in adaptability, and that gap explains both its strong performance and its remaining limits.

APPENDICES

A FINAL AGENT FLOWCHART

The figure below summarizes the final agent pipeline described in Section 1.

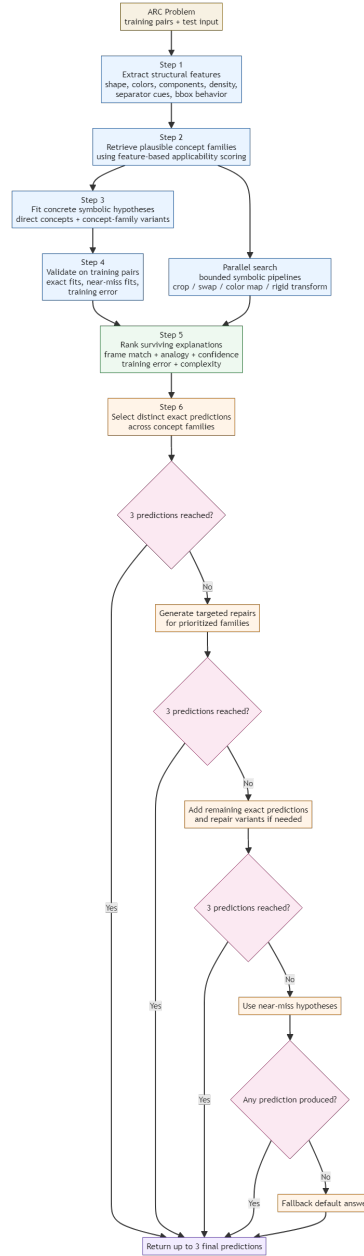


Figure 1—High-level flowchart of the final ARC-AGI agent pipeline, from feature extraction through concept retrieval, hypothesis fitting, validation, ranking, repair generation, and final prediction selection.

B GRADESCOPE RESULTS

The final Gradescope evaluation reported the following overall results:

- Training problems passed: 48
- Test problems passed: 45
- Total solved: 93 / 96
- Running time: 0:00:20.676419

By set, the hidden-test misses were:

- Set B: 28e73c20
- Set D: c8b7cc0f, 60a26a3e

C SUCCESSFUL PROBLEM PLOTS

The figures below provide the full training and test visualizations for the three successful problems discussed in Section 3.

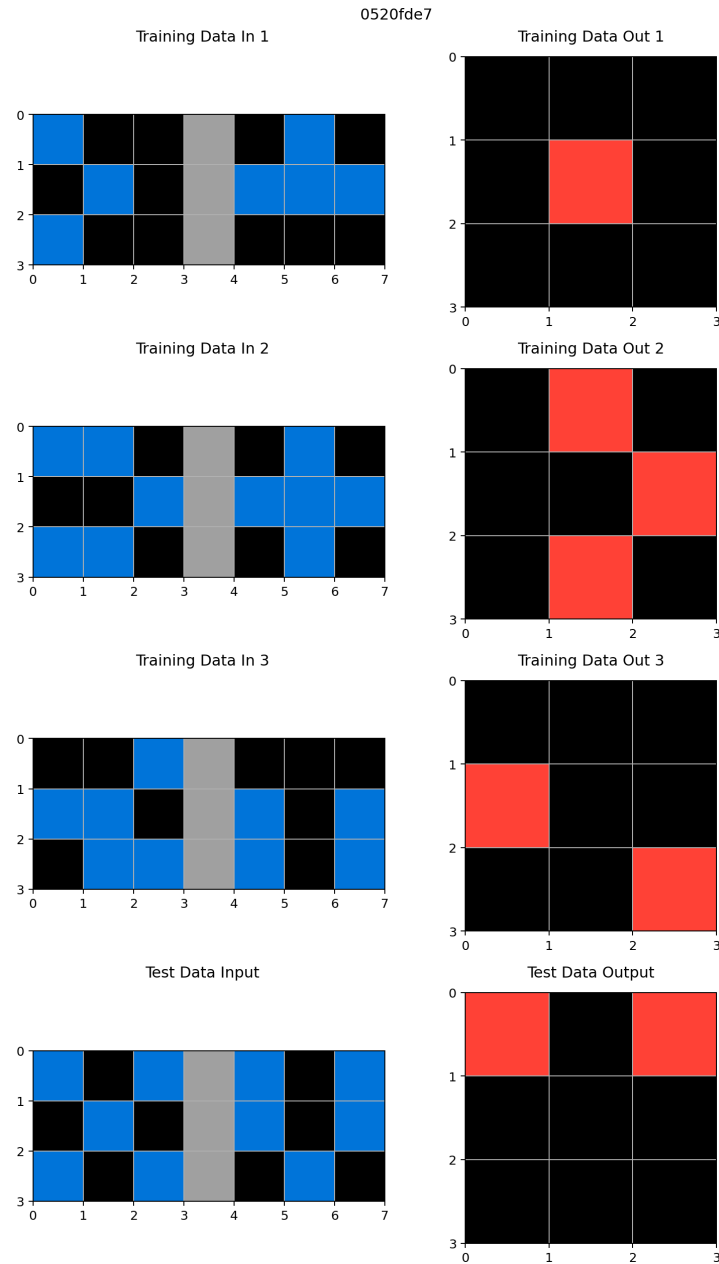


Figure 2—Problem 0520fde7. The visible examples show a uniform vertical separator and a reduced output that summarizes a Boolean relationship between the left and right panels.

62c24649

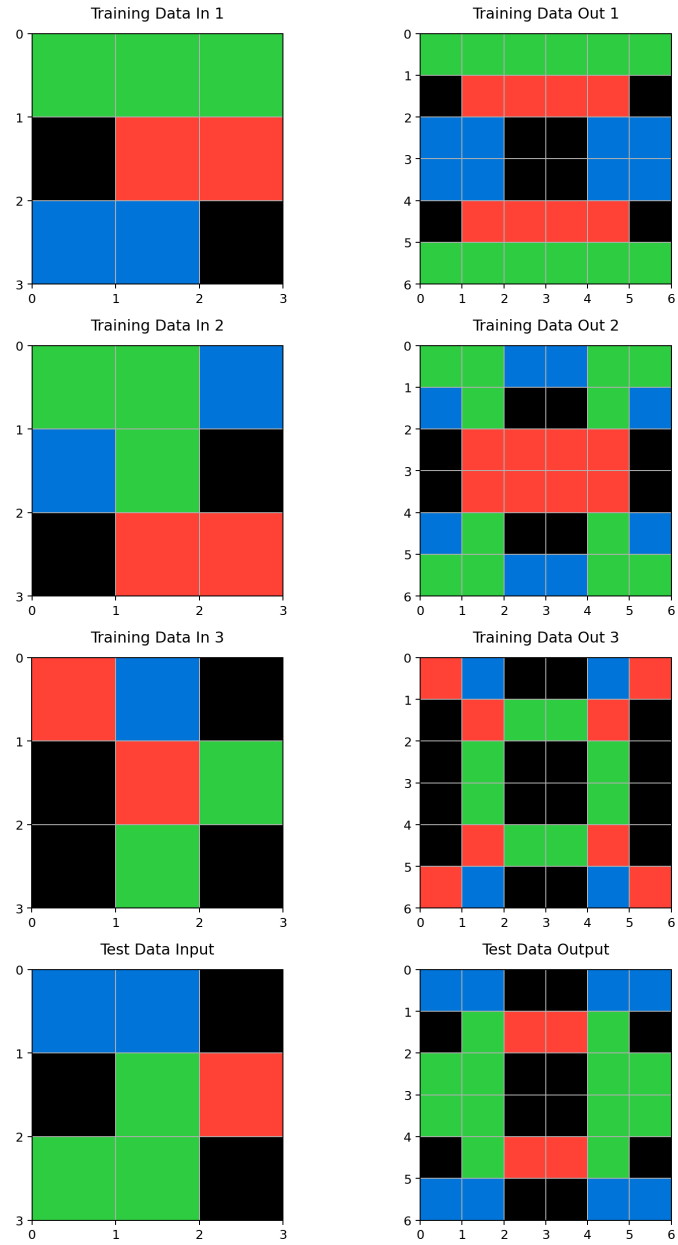


Figure 3—Problem 62c24649. Small seed patterns expand into larger symmetric outputs, which matches the agent's generative mirroring strategy.

e9b4f6fc

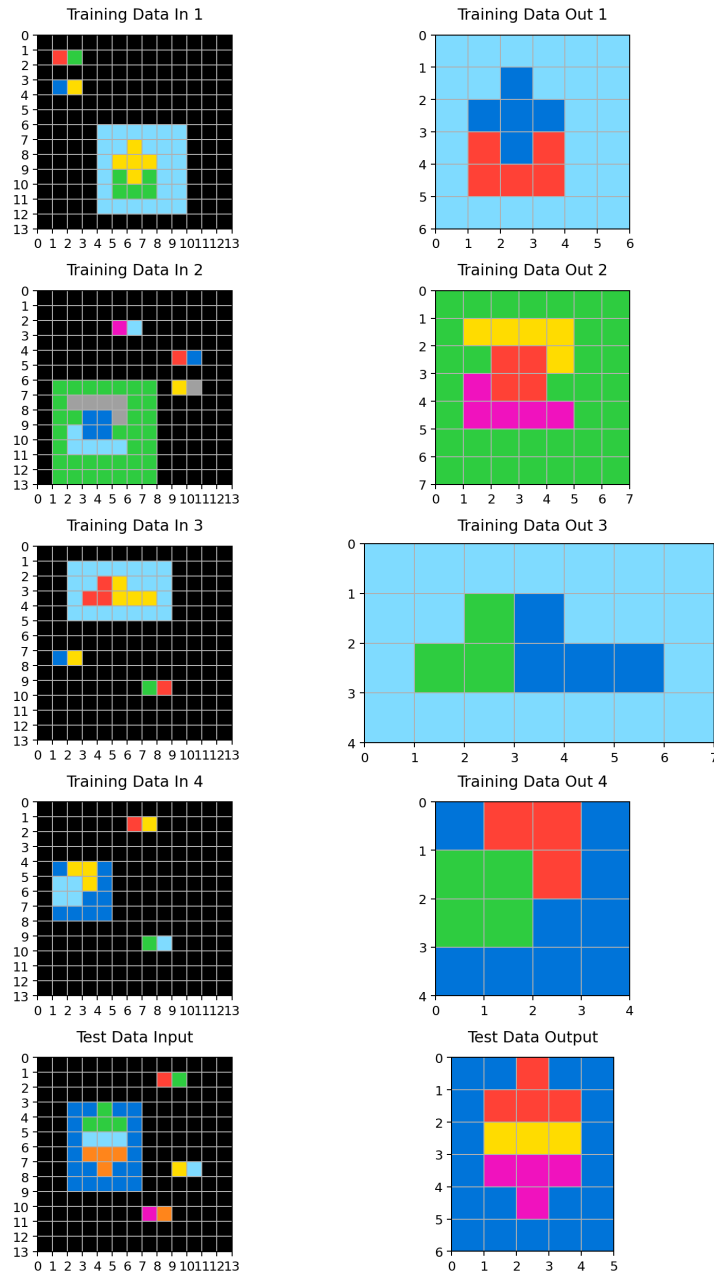


Figure 4—Problem e9b4f6fc. Detached marker pairs act as a legend that guides object selection, recoloring, and cropping in the final output.

D FAILURE PROBLEM PLOTS

The figures below provide the full training and test visualizations for the two hidden-test failures discussed in Section 4.

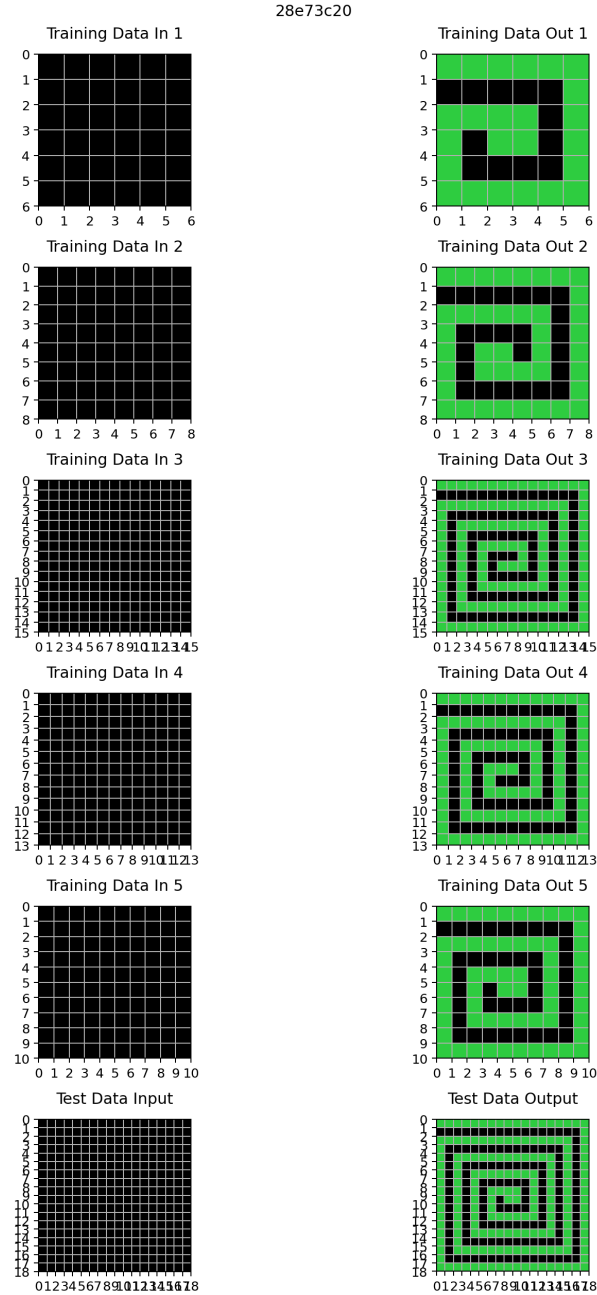


Figure 5—Problem 28e73c20. Empty visible inputs require the agent to infer a spiral-generation rule whose precise parameterization is only partially constrained by the training examples.

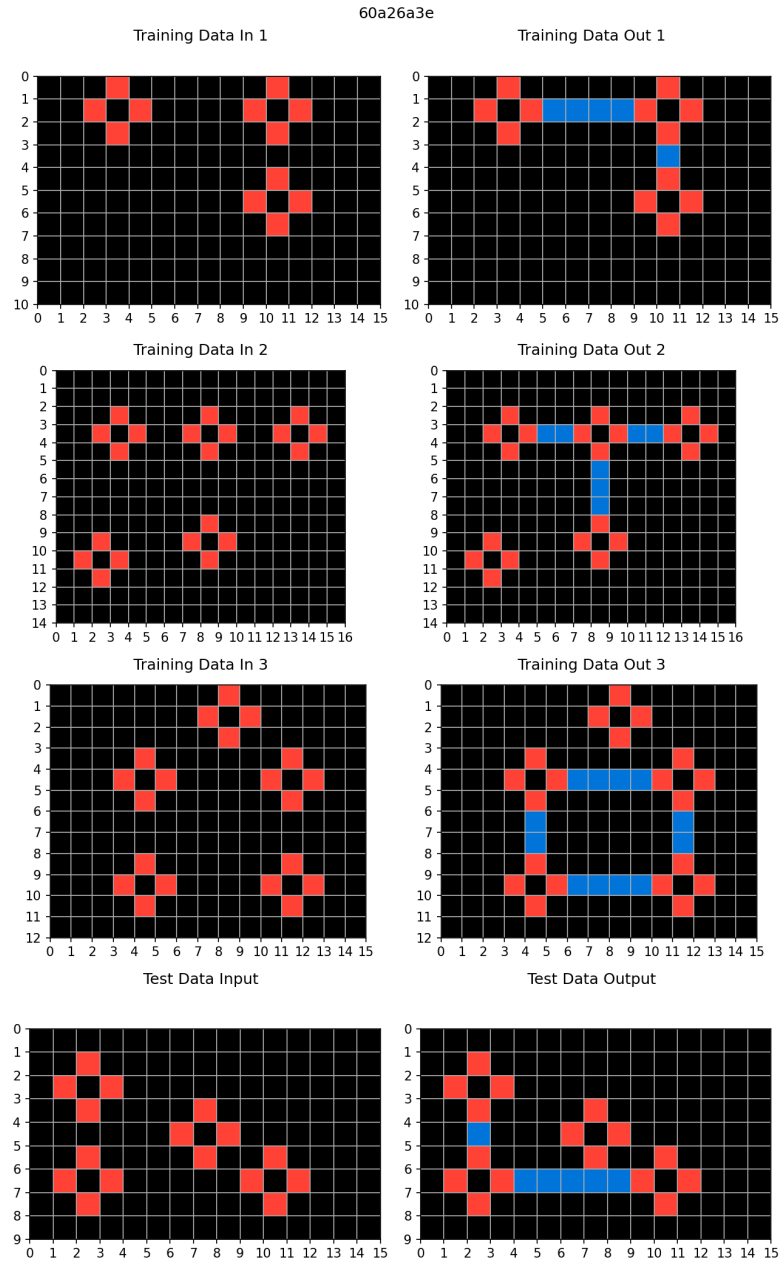


Figure 6—Problem 60a26a3e. Correct outputs connect selected diamond centers, but distractor shapes can imitate valid centers closely enough to trigger the wrong anchors.

E HOW THE RANKER WORKS

Before ranking begins, the concept registry performs a simpler retrieval step. The registry itself does not use a separate learned scoring model. Instead, each concept assigns a hand-written applicability score from the extracted structural features, such as whether the task preserves shape, shrinks to an extracted region, contains separator axes, or appears object-relational. The registry then sorts concepts by those applicability scores and passes only the positive-scoring ones into hypothesis fitting.

The ranker decides which fitted symbolic explanations should be trusted most after hypothesis generation. Its purpose is not only to reward candidates that reproduce the visible training pairs, but also to prefer explanations that look more structurally appropriate for the problem. Its role became especially important in the final agent because exact visible fits were not always enough to identify the explanation most likely to generalize.

Each fitted hypothesis enters ranking with several associated signals. The first is an explanation signature, which summarizes the candidate’s high-level assumptions about the task, including its shape behavior, its frame, and whether it relies on cues such as separator structure or bounding-box style extraction. The second is training error, which counts how many output cells across the training examples disagree with the candidate’s prediction. Exact fits therefore have zero training error, while near-miss hypotheses retain a positive residual error instead of being discarded immediately. The third is confidence, which is set highest for exact fits and decreases as training error increases. The fourth is complexity cost, which penalizes hypotheses that rely on more elaborate symbolic machinery when simpler alternatives are available.

The ranker also compares each explanation signature to the structural features computed earlier from the problem. The comparison produces a frame-match score. A candidate receives a higher score when its assumptions agree with the observed task structure. For example, it is rewarded if it preserves shape when the problem appears shape-preserving, uses separator logic when clear separator axes are present, uses a bounding-box style explanation when the output behaves like an extraction, or selects a frame that agrees with the problem classifier’s coarse structural judgment.

In addition, the agent applies a within-problem case-analogy score. Once the solver has found a strong explanation for the current task, it stores a signature of that explanation in case memory. Structurally similar candidates from the same problem can then receive a small bonus if their signatures align with the stored case and especially if they belong to the same conceptual family. The mechanism is intentionally local to one problem. Earlier cross-problem analogy behavior was removed because it made ranking less stable on hidden-style validation suites.

The ranker combines those signals into one diagnostic score. It also adds small family-level bonuses when the observed features support a selected concept family. In practical terms, the ranker favors candidates with better frame agreement, stronger analogy support, higher confidence, lower residual training error, and lower complexity cost. Candidates are then sorted so that the system prefers explanations that are both visibly correct on the training pairs and structurally plausible for the problem as a whole. Exact fits are still preferred over near-miss candidates by default, but near-miss candidates are retained because they can later support repair-oriented fallback when the top exact explanation turns out to be too brittle. In that sense, the ranker is the bridge between hypothesis generation and final prediction selection: it turns a large set of symbolic possibilities into a small ordered list of explanations that the agent is willing to trust.

F HOW REPAIR GENERATION WORKS

Repair generation is the mechanism the agent uses when one promising symbolic explanation appears to contain the right overall abstraction but still looks too brittle to trust as the only final answer. Instead of discarding that candidate completely, the agent treats it as a basis for a small local search. The goal is to keep the same high-level concept and vary one symbolic choice at a time.

In practice, the repair process is family-specific. Different concept families fail in different ways, so the agent does not apply one universal repair rule to everything. For center-connection tasks, it may vary the selected centers, row-only versus column-only linking, or small gap settings. For frame symmetry tasks, it may test alternate symmetry modes. For legend-guided extraction, it may change which object is treated as the main target or which legend mapping is applied. For marker-path, cone, spiral, motif-completion, enclosure-repair, and slide-to-border families, it similarly perturbs a small symbolic decision that often determines whether a nearly correct explanation generalizes or breaks.

Repair generation happens after ranking has identified strong exact candidates and plausible near-miss candidates. The agent first prefers diverse exact predictions across concept families. If that process leaves room in the final prediction set, or if a high-priority family appears especially fragile, the agent generates repaired outputs from one or two promising base hypotheses. The repaired outputs are added only if they produce genuinely distinct predictions.

The main reason for this mechanism is that many ARC tasks are not fully disambiguated by the visible examples. A top candidate may get the correct concept family but miss one local symbolic choice, such as the selected center, path routing decision, symmetry interpretation, or fill mode. Repair generation gives the agent a controlled way to explore those nearby variants without abandoning the original explanation.